

DATA STRUCTURES & ALGORITHMS WITH JAVA

UPDATED STRICT PROJECT-CHAT MCQ REVISION BOOK

180 MCQs • 90 Theory • 90 Code • Answers and explanations directly below every question

UPDATED PROJECT SCOPE

This edition is restricted to topics covered in this DSA-with-Java project. It includes algorithms and complexity; Big-O, Ω , Θ and case analysis; linear search; binary search; recursion and stack unwinding; stacks and balanced-bracket checking; binary trees; BST insertion/search/duplicates; inorder, preorder and postorder; recursive and iterative inorder traversal; traversal complexity; **singly linked lists; and doubly linked lists.**

LinkedList coverage: node structure, head/tail, traversal, search, insertion, deletion, edge cases, complexity, append, reverse concepts, and singly linked-list reference updates.

Doubly LinkedList coverage: prev/next references, forward/backward traversal, insertion/deletion, head/tail maintenance, middle-node deletion, link consistency, and single-node/empty-list cases.

Excluded: sorting, hashing, graphs, greedy algorithms, dynamic programming, queues/deques/priority queues, heaps as a separate topic, and unrelated topics not covered in this project.

TABLE OF CONTENTS

PART A — THEORY: 1–90

Easy: 1–30 • Medium: 31–60 • Hard: 61–90

PART B — CODE SNIPPETS: 91–180

Easy: 91–120 • Medium: 121–150 • Hard: 151–180

PART A — THEORY

LEVEL — EASY

1.

What is the main purpose of an algorithm?

- A. To provide ordered steps for solving a problem
- B. To store only primitive values
- C. To replace every data structure
- D. To compile Java code

Correct Answer: A

Explanation: An algorithm is a finite, ordered set of steps used to solve a problem.

2.

What does time complexity describe?

- A. How running time grows with input size
- B. How many classes a program has
- C. How much source code is written
- D. How many variables are declared

Correct Answer: A

Explanation: Time complexity describes the growth of computational work as input size increases.

3.

What does space complexity describe?

- A. How memory usage grows with input size
- B. How fast the CPU clock runs
- C. How many methods exist
- D. How many test cases are written

Correct Answer: A

Explanation: Space complexity measures how memory requirements grow with input size.

4.

Which notation commonly describes an asymptotic upper bound?

- A. Big-O
- B. Big- Ω only
- C. Big- Θ only
- D. JVM

Correct Answer: A

Explanation: Big-O gives an asymptotic upper bound on growth.

5.

Which notation commonly describes an asymptotic lower bound?

- A. Big- Ω
- B. Big-O
- C. Big- Θ only
- D. LIFO

Correct Answer: A

Explanation: Big- Ω represents an asymptotic lower bound.

6.

Which notation describes a tight asymptotic bound?

- A. Big- Θ
- B. Big-O only
- C. Big- Ω only
- D. FIFO

Correct Answer: A

Explanation: Big- Θ describes a tight asymptotic bound when both upper and lower bounds match.

7.

In linear search, where can the best case occur?

- A. At the first element
- B. Only at the last element
- C. Only when the array is empty
- D. Only in a sorted array

Correct Answer: A

Explanation: If the target is the first element, linear search stops immediately.

8.

What is the usual worst-case time complexity of linear search?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Linear search may inspect all n elements, giving $O(n)$ worst-case time.

9.

What condition is required for ordinary binary search?

- A. The data must be sorted
- B. The data must have no duplicates
- C. The length must be prime
- D. The data must be stored in a linked list

Correct Answer: A

Explanation: Binary search uses ordering to discard half of the remaining search space.

10.

What is the usual worst-case time complexity of binary search?

- A. $O(\log n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(1)$

Correct Answer: A

Explanation: The search interval is roughly halved at every step.

11.

What is a base case in recursion?

- A. A condition that stops further recursive calls
- B. A condition that always creates another call
- C. A loop inside a method
- D. A Java package

Correct Answer: A

Explanation: The base case prevents recursion from continuing indefinitely.

12.

What happens during recursion stack unwinding?

- A. Completed calls return in reverse order of their active-call nesting
- B. All calls execute simultaneously
- C. The heap is deleted
- D. The root of a tree changes

Correct Answer: A

Explanation: After the base case, recursive calls return from the deepest active call outward.

13.

Which operation adds an item to the top of a stack?

- A. push
- B. pop
- C. peek
- D. search

Correct Answer: A

Explanation: push inserts an element at the stack top.

14.

Which operation removes the top stack element?

- A. pop
- B. push
- C. peek
- D. root

Correct Answer: A

Explanation: pop removes and usually returns the top element.

15.

Which operation reads the top stack element without removing it?

- A. peek
- B. push
- C. pop
- D. insert

Correct Answer: A

Explanation: peek observes the top element without removing it.

16.

Which principle does a stack follow?

- A. LIFO
- B. FIFO
- C. Random-only access
- D. Sorted order

Correct Answer: A

Explanation: Stack means Last In, First Out.

17.

What is the root of a binary tree?

- A. The topmost node
- B. The leftmost leaf only
- C. The deepest node
- D. Any node with two children

Correct Answer: A

Explanation: The root is the topmost node of a tree.

18.

What is the maximum number of children of a binary-tree node?

- A. 2
- B. 1
- C. 3
- D. Unlimited

Correct Answer: A

Explanation: A binary-tree node has at most two children: left and right.

19.

What is a leaf node?

- A. A node with no children
- B. A node with two children
- C. The root only
- D. A node with exactly one child

Correct Answer: A

Explanation: A leaf has neither a left nor a right child.

20.

What property distinguishes a BST from an ordinary binary tree?

- A. Keys are ordered relative to each node
- B. Every node must have two children
- C. The tree must be complete
- D. Duplicates are always forbidden by the definition

Correct Answer: A

Explanation: A BST maintains an ordering relationship between left and right subtrees and the node.

21.

Which traversal visits left subtree, node, then right subtree?

- A. Inorder
- B. Preorder
- C. Postorder
- D. Level-order

Correct Answer: A

Explanation: Inorder follows Left → Node → Right.

22.

Which traversal visits node before its children?

- A. Preorder
- B. Inorder
- C. Postorder
- D. Reverse-order

Correct Answer: A

Explanation: Preorder follows Node → Left → Right.

23.

Which traversal visits the node after both subtrees?

- A. Postorder
- B. Preorder
- C. Inorder
- D. Root-order

Correct Answer: A

Explanation: Postorder follows Left → Right → Node.

24.

Why can an explicit stack be used for iterative inorder traversal?

- A. It simulates the recursive call stack
- B. It sorts the tree
- C. It stores only leaf nodes
- D. It changes the BST property

Correct Answer: A

Explanation: The explicit stack stores nodes whose recursive calls would otherwise remain pending.

25.

What is the usual time complexity of traversing every node of a tree?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Correct Answer: A

Explanation: A complete traversal visits each of n nodes once.

26.

In a singly linked list, what does a node normally contain?

- A. Data and a reference to the next node
- B. Only data
- C. Two child references
- D. Only a previous reference

Correct Answer: A

Explanation: A typical singly linked-list node stores data and a next reference.

27.

What does the head of a linked list refer to?

- A. The first node
- B. The last node
- C. The middle node
- D. The largest node

Correct Answer: A

Explanation: head points to the first node of the list.

28.

What does the next reference of the last node in a normal singly linked list contain?

- A. null
- B. head always
- C. The previous node
- D. The root

Correct Answer: A

Explanation: The last node has no successor, so its next reference is null.

29.

What is a major advantage of linked-list insertion at the beginning?

- A. It can be $O(1)$ when the head is known
- B. It always requires sorting
- C. It requires binary search
- D. It always requires $O(n^2)$

Correct Answer: A

Explanation: Changing the head and one next reference can insert a node at the beginning in constant time.

30.

What extra link does a doubly linked-list node normally have compared with a singly linked-list node?

- A. A previous reference
- B. A root reference
- C. A parent array
- D. A stack reference

Correct Answer: A

Explanation: A doubly linked-list node normally stores both prev and next references.

LEVEL — MEDIUM

31.

For linear search on n elements, what is the best-case time complexity?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: If the first element matches, only one comparison is needed.

32.

For binary search, what happens to the search interval after one typical step?

- A. It is approximately halved
- B. It doubles
- C. It becomes n^2
- D. It is always reduced by exactly one

Correct Answer: A

Explanation: Binary search discards roughly half the remaining ordered search space.

33.

If an algorithm performs $3n + 7$ basic operations, its asymptotic time complexity is:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. $O(1)$

Correct Answer: A

Explanation: Constant factors and lower-order terms are ignored asymptotically, leaving linear growth.

34.

If a recursive traversal visits every tree node once, its time complexity is:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Each node is processed once, so total work is linear.

35.

What is the auxiliary space of recursive inorder traversal in a highly skewed tree?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log 1)$
- D. $O(n^2)$

Correct Answer: A

Explanation: A skewed tree can create n nested recursive calls, requiring $O(n)$ call-stack space.

36.

Why can iterative inorder avoid recursive call-stack usage?

- A. It stores pending nodes in an explicit stack
- B. It never stores nodes
- C. It uses binary search
- D. It converts the tree into an array

Correct Answer: A

Explanation: The programmer-managed stack replaces the implicit recursion stack.

37.

If a stack is empty, what does pop conceptually attempt to do?

- A. Remove a nonexistent top element
- B. Insert a new element
- C. Read the root
- D. Traverse a tree

Correct Answer: A

Explanation: Popping an empty stack is an underflow condition and must be handled appropriately.

38.

For balanced-bracket checking, what should happen when a closing bracket is encountered?

- A. It should match the most recent unmatched opening bracket
- B. It should always be pushed
- C. It should be ignored
- D. It should replace the first opening bracket

Correct Answer: A

Explanation: The most recent opening bracket must be checked first, which is exactly the stack's LIFO behavior.

39.

In a BST, where is a value smaller than the current node normally placed during insertion?

- A. Left subtree
- B. Right subtree
- C. Root only
- D. Outside the tree

Correct Answer: A

Explanation: The BST ordering convention places smaller values in the left subtree.

40.

In a BST, where is a value greater than the current node normally placed?

- A. Right subtree
- B. Left subtree
- C. Always at root
- D. It cannot be inserted

Correct Answer: A

Explanation: The usual BST rule places greater values in the right subtree.

41.

What should an implementation decide explicitly when inserting a duplicate BST key?

- A. A duplicate policy such as left, right, count, or ignore
- B. Nothing; duplicates automatically have one universal location
- C. Always delete the root
- D. Convert the tree to a linked list

Correct Answer: A

Explanation: BST implementations need a defined duplicate policy; there is no single mandatory universal rule.

42.

In a BST, what does inorder traversal produce when keys obey the BST ordering?

- A. Keys in sorted order
- B. Keys in insertion order always
- C. Only leaf keys
- D. Reverse insertion order always

Correct Answer: A

Explanation: Inorder traversal of a BST visits keys in nondecreasing order under the usual ordering.

43.

For the insertion sequence 10, 5, 2, 4, where is 4 inserted relative to 5?

- A. In 5's right subtree, under node 2
- B. As the right child of 10
- C. As the left child of 10
- D. As the left child of 2

Correct Answer: A

Explanation: $4 < 5$ so go left; $4 > 2$ so it becomes the right child of 2.

44.

In the sequence 10, 5, 2, 4, 20, 30, what is the parent of 4?

- A. 2
- B. 5
- C. 10
- D. 20

Correct Answer: A

Explanation: 4 is less than 10, less than 5, but greater than 2, so it becomes 2's right child.

45.

For a linked list with a head reference, traversing from head to null is:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Each node is followed once, so traversal is linear.

46.

Why is random access slower in a linked list than in an array?

- A. Nodes are reached by following links sequentially
- B. Linked lists cannot store integers
- C. Arrays always use recursion
- D. Linked lists are always sorted

Correct Answer: A

Explanation: A linked list does not provide direct indexing; reaching position i generally requires following links.

47.

If a singly linked list inserts at the head when head is available, the pointer/reference update is generally:

- A. `newNode.next = head; head = newNode`
- B. `head.next = newNode; newNode = head`
- C. `head = null; newNode.next = null`
- D. `newNode.prev = head`

Correct Answer: A

Explanation: The new node points to the old head, then head is changed to the new node.

48.

What is the usual time complexity of deleting the first node of a singly linked list when head is known?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Only the head reference needs to be advanced to the next node.

49.

Why can a doubly linked list delete a known node efficiently without finding its predecessor?

- A. The node has a prev reference
- B. It uses binary search
- C. It stores nodes in an array
- D. It has no next reference

Correct Answer: A

Explanation: The prev link directly identifies the predecessor, reducing the need for a separate predecessor search.

50.

What must be updated when deleting a middle node from a doubly linked list?

- A. Both neighboring links around the removed node
- B. Only the data value
- C. Only the head, always
- D. Only the removed node's data

Correct Answer: A

Explanation: The predecessor's next and successor's prev must be connected around the removed node.

51.

What is the extra reference stored per node in a doubly linked list compared with a singly linked list?

- A. One additional reference, prev
- B. Two arrays
- C. One stack
- D. One root pointer

Correct Answer: A

Explanation: Doubly linked nodes normally have both prev and next, adding one link field.

52.

For a list of n nodes, searching for a value without extra indexing is usually:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: In the worst case, each node may need to be checked.

53.

What is a common edge case for linked-list operations?

- A. An empty list
- B. A list must always have 100 nodes
- C. The head must be an integer
- D. The list must be sorted

Correct Answer: A

Explanation: Empty-list handling is essential for insertion, deletion, search, and traversal.

54.

What should happen to head after deleting the only node in a singly linked list?

- A. head should become null
- B. head should point to the deleted node
- C. head should point to itself
- D. head should become 1

Correct Answer: A

Explanation: With no nodes remaining, the head reference must be null.

55.

In a doubly linked list with head and tail references, what should happen after deleting the only node?

- A. Both head and tail should become null
- B. Only head becomes null
- C. Only tail becomes null
- D. Both point to the deleted node

Correct Answer: A

Explanation: An empty doubly linked list has neither a first nor a last node.

56.

What is the main traversal-direction benefit of a doubly linked list?

- A. It can move forward and backward using next and prev
- B. It can always access any index in $O(1)$
- C. It automatically sorts values
- D. It eliminates null references

Correct Answer: A

Explanation: The prev link permits backward traversal.

57.

What does a linked-list node reference store?

- A. A reference to another node object
- B. The entire list copied into the node
- C. A CPU instruction
- D. A Java package

Correct Answer: A

Explanation: The link is a Java object reference pointing to another node or null.

58.

What is the usual space complexity of storing n linked-list nodes?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Each node requires storage, so total storage grows linearly.

59.

If a linked list is empty and an insertion creates its first node, what commonly happens?

- A. head is set to the new node
- B. head must remain null
- C. The new node is discarded
- D. The list becomes a BST

Correct Answer: A

Explanation: The first node becomes the head of the previously empty list.

60.

What is the main trade-off of a doubly linked list versus a singly linked list?

- A. More memory and pointer updates in exchange for backward traversal and easier known-node deletion
- B. Less memory and fewer references
- C. Automatic sorting
- D. $O(1)$ random access

Correct Answer: A

Explanation: The prev link costs memory and update work but enables backward navigation and simplifies certain deletions.

LEVEL — HARD

61.

An algorithm takes $n/2 + 20$ operations. Which asymptotic class best describes it?

- A. $\Theta(n)$
- B. $\Theta(\log n)$
- C. $\Theta(n^2)$
- D. $\Theta(1)$

Correct Answer: A

Explanation: The dominant term is $n/2$, so the growth is linear; constants do not change the Θ class.

62.

A linear search finds the target at position n in a zero-based array of n elements. Its comparison count is:

- A. n
- B. $n-1$
- C. 1
- D. $\log n$

Correct Answer: A

Explanation: The last position requires checking all n elements.

63.

A binary search repeatedly halves a search interval. After k halvings, the remaining size is approximately:

- A. $n / 2^k$
- B. $n + 2k$
- C. n^k
- D. $2n$

Correct Answer: A

Explanation: Each halving divides the current interval by two.

64.

Why can recursive tree traversal use $O(n)$ stack space even though its total time is $O(n)$?

- A. A skewed tree can have n nested active calls
- B. Every node creates n^2 calls
- C. Time and space are always identical
- D. The tree is stored twice

Correct Answer: A

Explanation: The total number of visits can be linear while the maximum recursion depth reaches n .

65.

A balanced BST has height $O(\log n)$. A search following one root-to-leaf path therefore takes:

- A. $O(\log n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(1)$

Correct Answer: A

Explanation: BST search follows one path whose length is bounded by the tree height.

66.

A completely skewed BST can have height:

- A. $O(n)$
- B. $O(\log n)$ always
- C. $O(1)$
- D. $O(n^2)$

Correct Answer: A

Explanation: If each node has only one child, the tree becomes chain-like with height proportional to n .

67.

For a BST containing distinct keys, if inorder prints 2 4 5 10 20 30, which statement is guaranteed?

- A. The sequence is sorted in ascending order
- B. The tree must be perfectly balanced
- C. The insertion order was 2 4 5 10 20 30
- D. Every node has two children

Correct Answer: A

Explanation: Inorder traversal of a valid BST with distinct keys is sorted, regardless of shape or insertion order.

68.

When recursively computing inorder traversal, why is the current node printed after the left recursive call?

- A. That ordering implements Left → Node → Right
- B. It prevents all recursion
- C. It makes the tree balanced
- D. It changes the node's value

Correct Answer: A

Explanation: The position of the print statement determines traversal order.

69.

An iterative inorder algorithm pushes a chain of left children before processing a node. Why?

- A. Those nodes must be processed after their left descendants but before appropriate right subtrees
- B. To sort their values numerically
- C. To delete them
- D. To guarantee a complete tree

Correct Answer: A

Explanation: The stack preserves nodes waiting for their left subtrees to finish.

70.

Suppose a stack-based balanced-bracket checker sees a closing bracket when the stack is empty. The expression is:

- A. Unbalanced
- B. Balanced
- C. Automatically corrected
- D. A valid BST

Correct Answer: A

Explanation: There is no opening bracket available to match the closing bracket.

71.

In a singly linked list, inserting after a known node x generally requires:

- A. Changing x.next and the new node's next reference
- B. Finding the root of a tree
- C. Changing every node
- D. Binary searching the list

Correct Answer: A

Explanation: The new node is linked between x and x's former successor.

72.

In a singly linked list, deleting a known node is not always $O(1)$ if only that node reference is available because:

- A. Its predecessor may be needed to bypass it
- B. The node has two parents
- C. Linked lists require sorting
- D. The node has no data

Correct Answer: A

Explanation: Without a predecessor or special technique, the previous node must be found to update its next link.

73.

In a doubly linked list, deleting a known middle node x can be done by conceptually setting:

- A. $x.prev.next = x.next$ and $x.next.prev = x.prev$
- B. $x.prev = \text{head}$ only
- C. $x.next = \text{root}$ only
- D. $\text{head} = x.next$ only

Correct Answer: A

Explanation: Both adjacent nodes must be linked to each other around x.

74.

If x is the head of a doubly linked list and has a next node, deleting x requires the new head's:

- A. prev to become null
- B. next to become null
- C. data to become zero
- D. tail to become null

Correct Answer: A

Explanation: The new first node must have no predecessor.

75.

If x is the tail of a doubly linked list and has a previous node, deleting x requires the new tail's:

- A. next to become null
- B. prev to become null
- C. data to become zero
- D. head to become null

Correct Answer: A

Explanation: The new last node must have no successor.

76.

A linked-list insertion at position i requires reaching position i-1. Without extra indexing, the traversal component is typically:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Reaching an arbitrary position requires sequentially following links.

77.

Why does a linked list not automatically gain binary-search complexity just because its values are sorted?

- A. Finding the middle node itself is not constant-time random access
- B. Sorted data cannot be searched
- C. Linked lists have no nodes
- D. Binary search always requires arrays by language rule

Correct Answer: A

Explanation: Binary search relies on efficient access to the midpoint; linked lists require traversal to reach it.

78.

For a doubly linked list, what is the conceptual invariant for a middle node x?

- A. If x.prev exists, $x.prev.next == x$; if x.next exists, $x.next.prev == x$
- B. x.prev must always be null
- C. x.next must always be null
- D. x must be the head

Correct Answer: A

Explanation: The forward and backward links should consistently point back to each other.

79.

Which failure most directly breaks a doubly linked list during insertion?

- A. Updating next but forgetting the corresponding prev link
- B. Using an integer for data
- C. Printing the list
- D. Checking head for null

Correct Answer: A

Explanation: Both directions must remain consistent after pointer/reference changes.

80.

A list has head and tail references. Appending to the tail can be $O(1)$ when:

- A. The tail reference is maintained
- B. The list is always sorted
- C. The list has no head
- D. The nodes contain integers

Correct Answer: A

Explanation: A tail reference gives direct access to the last node.

81.

A singly linked list stores only head, not tail. Appending to the end is typically:

- A. $O(n)$
- B. $O(1)$ always
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Without a tail reference, the last node must generally be found by traversal.

82.

If a singly linked list also maintains tail, appending a node can be:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. Impossible

Correct Answer: A

Explanation: The tail reference directly identifies where the new node is attached.

83.

What is the key reason linked-list insertion can be efficient after a known position?

- A. Only a constant number of references need to be changed
- B. All nodes must be shifted
- C. The list must be sorted
- D. The CPU accesses all nodes at once

Correct Answer: A

Explanation: Unlike arrays, linked lists do not require shifting a suffix of elements.

84.

What is a correct relationship for a nonempty doubly linked list's head?

- A. `head.prev == null`
- B. `head.prev == tail`
- C. `head.next == null` always
- D. head must have two children

Correct Answer: A

Explanation: The first node has no predecessor.

85.

What is a correct relationship for a nonempty doubly linked list's tail?

- A. `tail.next == null`
- B. `tail.next == head` always
- C. `tail.prev == null` always
- D. tail must have two children

Correct Answer: A

Explanation: The last node has no successor.

86.

If a singly linked list contains one node x, what must be true?

- A. `x.next == null` and `head == x`
- B. `x.next == x` always
- C. `head == null`
- D. x must have a prev field

Correct Answer: A

Explanation: A one-node ordinary singly linked list has the node as head and no successor.

87.

If a doubly linked list contains one node x and uses head/tail, what must be true?

- A. `head == tail == x`, `x.prev == null`, `x.next == null`
- B. `head == null`
- C. `x.prev == x`
- D. `x.next` must point to head

Correct Answer: A

Explanation: The single node is both first and last and has no neighbors.

88.

Which operation differs most fundamentally between a singly and doubly linked list?

- A. Backward traversal is directly supported by the doubly linked list
- B. Both can traverse forward
- C. Both can store data
- D. Both can have a head

Correct Answer: A

Explanation: The additional prev link enables direct backward traversal.

89.

If a linked-list operation accidentally loses the only reference to the first node, what is the main consequence?

- A. The list may become unreachable from the program
- B. The list automatically reconstructs itself
- C. The tree becomes balanced
- D. Binary search becomes faster

Correct Answer: A

Explanation: Without a reachable head/reference chain, the nodes may no longer be accessible through the list.

90.

If a tree has n nodes, what is the total number of node-processing operations in a traversal that visits each node exactly once?

- A. n
- B. $\log n$
- C. n^2
- D. 1

Correct Answer: A

Explanation: Each of the n nodes is processed once, so the total number of node-processing operations is n .

PART B — CODE SNIPPETS

LEVEL — EASY

91.

What is printed?

```
int[] a = {4, 7, 2};  
System.out.println(a[0]);
```

- A. 4
- B. 7
- C. 2
- D. 0

Correct Answer: A

Explanation: Index 0 refers to the first element, 4.

92.

What is the result?

```
int[] a = {4, 7, 2};  
int target = 7;  
int i = 0;  
while (i < a.length && a[i] != target) i++;  
System.out.println(i);
```

- A. 1
- B. 0
- C. 2
- D. 3

Correct Answer: A

Explanation: The target 7 is at index 1.

93.

What is the result?

```
int[] a = {1, 3, 5, 7};  
int low = 0, high = a.length - 1;  
int mid = (low + high) / 2;  
System.out.println(a[mid]);
```

- A. 3
- B. 1
- C. 5
- D. 7

Correct Answer: A

Explanation: mid is $(0+3)/2 = 1$, so `a[1]` is 3.

94.

What is printed?

```
static void f(int n) {  
    if (n == 0) return;  
    System.out.print(n + " ");  
    f(n - 1);  
}
```

- A. 3 2 1
- B. 1 2 3
- C. 3 2 1 0
- D. 2 1 0

Correct Answer: A

Explanation: The value is printed before the recursive call, producing 3, 2, 1.

95.

What is printed?

```
Stack<Integer> s = new Stack<>();  
s.push(10);  
s.push(20);  
System.out.println(s.peek());
```

- A. 20
- B. 10
- C. 30
- D. null

Correct Answer: A

Explanation: 20 is the top element and peek does not remove it.

96.

What is printed?

```
Stack<Integer> s = new Stack<>();  
s.push(10);  
s.push(20);  
System.out.println(s.pop());
```

- A. 20
- B. 10
- C. 30
- D. null

Correct Answer: A

Explanation: pop removes and returns the most recently pushed element, 20.

97.

What does this return for ``{[()]}``?

```
isBalanced("{[()]}")
```

- A. true
- B. false
- C. null
- D. Compilation error

Correct Answer: A

Explanation: Each closing bracket matches the most recent opening bracket.

98.

Given this node:

```
class Node {  
    int data;  
    Node next;  
}
```

Which statement correctly creates a node with data 10?

- A. Node n = new Node(); n.data = 10;
- B. Node n = new Node(10); always
- C. Node n = 10;
- D. Node n = null(10);

Correct Answer: A

Explanation: The no-argument constructor is used, then data is assigned.

99.

After:

```
Node n = new Node();  
n.data = 10;  
n.next = null;
```

what does `n.next` contain?

- A. null
- B. 10
- C. n.data
- D. head

Correct Answer: A

Explanation: The code explicitly assigns null to next.

100.

What does this print?

```
Node head = null;
System.out.println(head == null);
```

- A. true
- B. false
- C. 0
- D. null

Correct Answer: A

Explanation: head is assigned null, so the comparison is true.

101.

Which code inserts `newNode` at the beginning of a singly linked list?

```
Node newNode = new Node();

A. newNode.next = head; head = newNode;
B. head.next = newNode;
C. newNode = head.next;
D. head = null;
```

Correct Answer: A

Explanation: The new node first points to the old head, then becomes the new head.

102.

For:

```
Node p = head;
while (p != null) {
    System.out.print(p.data + " ");
    p = p.next;
}
```

what is the loop's purpose?

- A. Traverse the list from head to the end
- B. Delete every node
- C. Reverse the list
- D. Find the tree root

Correct Answer: A

Explanation: p follows next references until it reaches null.

103.

What is printed?

```
Node a = new Node(); a.data = 5;
Node b = new Node(); b.data = 8;
a.next = b;
System.out.println(a.next.data);
```

- A. 8
- B. 5
- C. null
- D. 0

Correct Answer: A

Explanation: a.next refers to b, whose data is 8.

104.

For a doubly linked node:

```
class Node {
    int data;
    Node prev, next;
}
```

which field points backward?

- A. prev
- B. next
- C. data
- D. head

Correct Answer: A

Explanation: prev points to the predecessor node.

105.

Which statement correctly links `a` before `b` in a doubly linked list when both exist?

- A. a.next = b; b.prev = a;
- B. a.prev = b; b.next = a;
- C. a.next = null; b.prev = null;
- D. a = b.next;

Correct Answer: A

Explanation: Forward and backward links must agree: a.next points to b and b.prev points to a.

106.

What is printed?

```
Node a = new Node();
a.data = 4;
Node b = new Node();
b.data = 9;
a.next = b;
b.prev = a;
System.out.println(b.prev.data);
```

- A. 4
- B. 9
- C. null
- D. 0

Correct Answer: A

Explanation: b.prev refers to a, whose data is 4.

107.

If `head` points to the first node and the list has one node, which is correct for a normal singly linked list?

- A. head.next == null
- B. head.next == head
- C. head == null
- D. head.prev == head

Correct Answer: A

Explanation: The only node has no successor.

108.

What is printed?

```
Node head = null;
if (head == null)
    System.out.println("empty");
```

- A. empty
- B. full
- C. null
- D. 0

Correct Answer: A

Explanation: The condition is true because head is null.

109.

Which statement removes the first node of a singly linked list?

- A. head = head.next;
- B. head.next = head;
- C. head = null.next;
- D. head.prev = head.next;

Correct Answer: A

Explanation: Moving head to its next node removes the old first node from the list chain.

110.

What does `tail.next = null;` normally represent when `tail` is the last node of a singly linked list?

- A. The tail has no successor
- B. The tail is the head
- C. The list is necessarily empty
- D. The tail is deleted automatically

Correct Answer: A

Explanation: The last node's next reference is null.

111.

Which traversal order is implemented?

```
visit(root);  
traverse(root.left);  
traverse(root.right);
```

- A. Preorder
- B. Inorder
- C. Postorder
- D. Binary search

Correct Answer: A

Explanation: The node is visited before left and right subtrees: Node → Left → Right.

112.

Which traversal order is implemented?

```
traverse(root.left);  
visit(root);  
traverse(root.right);
```

- A. Inorder
- B. Preorder
- C. Postorder
- D. Reverse level order

Correct Answer: A

Explanation: The node is visited between left and right subtrees.

113.

Which traversal order is implemented?

```
traverse(root.left);  
traverse(root.right);  
visit(root);
```

- A. Postorder
- B. Preorder
- C. Inorder
- D. Search order

Correct Answer: A

Explanation: The node is visited after both subtrees.

114.

In recursive inorder, what should happen first for a non-null node?

```
void inorder(Node n) {  
    // ?  
}
```

- A. inorder(n.left)
- B. print n.data
- C. inorder(n.right)
- D. return immediately

Correct Answer: A

Explanation: Inorder requires the left subtree before visiting the node.

115.

What does this base case do?

```
if (current == null) return;
```

- A. Stops recursion for an empty subtree
- B. Deletes the tree
- C. Creates a new node
- D. Pushes the node

Correct Answer: A

Explanation: A null child represents an empty subtree, so recursion stops there.

116.

Which structure commonly replaces the recursive call stack in iterative inorder traversal?

- A. An explicit stack
- B. A queue only
- C. A second tree
- D. A primitive int

Correct Answer: A

Explanation: An explicit stack stores nodes whose processing is pending.

117.

What is printed?

```
Node head = new Node();  
head.data = 3;  
head.next = null;  
System.out.println(head.data);
```

- A. 3
- B. null
- C. 0
- D. 1

Correct Answer: A

Explanation: head.data was assigned 3.

118.

For a doubly linked list with one node `x`, which assignment pair represents an empty predecessor/successor?

- A. x.prev = null; x.next = null;
- B. x.prev = x; x.next = x;
- C. x.prev = head; x.next = tail;
- D. x.prev = 0; x.next = 0;

Correct Answer: A

Explanation: A single node has no previous or next node.

119.

Which line advances a singly linked-list traversal by one node?

- A. current = current.next;
- B. current = current.prev;
- C. current++;
- D. next = head;

Correct Answer: A

Explanation: Following current.next moves to the next node.

120.

What is printed?

```
Node x = new Node();  
x.data = 12;  
System.out.println(x.data);
```

- A. 12
- B. 0
- C. null
- D. 1

Correct Answer: A

Explanation: The data field is explicitly assigned the value 12.

LEVEL — MEDIUM

121.

What is printed?

```
int[] a = {2,4,6,8,10};
int low=0, high=4;
while(low<=high){
    int mid=(low+high)/2;
    if(a[mid]==8){ System.out.println(mid); break; }
    if(a[mid]<8) low=mid+1;
    else high=mid-1;
}
```

- A. 3
- B. 2
- C. 4
- D. 1

Correct Answer: A

Explanation: 8 is at index 3.

122.

What is printed?

```
static int f(int n){
    if(n==0) return 0;
    return n + f(n-1);
}
System.out.println(f(4));
```

- A. 10
- B. 4
- C. 6
- D. 0

Correct Answer: A

Explanation: The result is $4+3+2+1+0 = 10$.

123.

What is printed?

```
static void f(int n){
    if(n==0) return;
    f(n-1);
    System.out.print(n+" ");
}
f(3);
```

- A. 1 2 3
- B. 3 2 1
- C. 0 1 2 3
- D. 2 1 3

Correct Answer: A

Explanation: The print occurs while recursion unwinds, so 1, 2, then 3 are printed.

124.

What is the top after this code?

```
Stack<Integer> s = new Stack<>();
s.push(1); s.push(2); s.push(3);
s.pop();
System.out.println(s.peek());
```

- A. 2
- B. 3
- C. 1
- D. null

Correct Answer: A

Explanation: 3 is popped, leaving 2 at the top.

125.

For a balanced-bracket algorithm, what should happen for `([])` at the `)`?

- A. It should be rejected because `[` is the most recent opening bracket
- B. It should pop `[` anyway
- C. It should push `)`
- D. It should ignore the mismatch

Correct Answer: A

Explanation: The top is `[`, so `)` does not match the most recent opening bracket.

126.

Consider:

```
Node newNode = new Node();
newNode.data = 7;
newNode.next = head;
head = newNode;
```

What is the time complexity of this insertion?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: Only a constant number of references are changed.

127.

Given:

```
Node current = head;
while(current.next != null)
    current = current.next;
```

What does `current` reference after the loop?

- A. The last node
- B. The first node
- C. The second node
- D. null

Correct Answer: A

Explanation: The loop stops when `current.next` is null, so `current` is the last node.

128.

What is the issue with this deletion if `head` is already null?

```
head = head.next;
```

- A. It attempts to dereference null
- B. It creates a new node
- C. It reverses the list
- D. It sorts the list

Correct Answer: A

Explanation: If `head` is null, accessing `head.next` causes a null dereference.

129.

What is printed?

```
Node a = new Node(); a.data=1;
Node b = new Node(); b.data=2;
Node c = new Node(); c.data=3;
a.next=b; b.next=c;
Node p=a;
while(p!=null){ System.out.print(p.data+" "); p=p.next; }
```

- A. 1 2 3
- B. 3 2 1
- C. 1 3 2
- D. 2 3 1

Correct Answer: A

Explanation: `p` follows `a → b → c → null`.

130.

Which code correctly inserts `x` after `a` in a singly linked list?

- A. `x.next=a.next; a.next=x;`
- B. `a.next=x.next; x.next=a;`
- C. `x=a.next; a=x;`
- D. `a.prev=x;`

Correct Answer: A

Explanation: x first saves a's old successor, then a points to x.

131.

For doubly linked insertion of x between a and b, which sequence is conceptually correct?

- A. `x.prev=a; x.next=b; a.next=x; b.prev=x;`
- B. `a.prev=x; b.next=x; x.next=a; x.prev=b;`
- C. `x.next=null; x.prev=null;`
- D. `a=x; b=x;`

Correct Answer: A

Explanation: All four links needed for the local connection are updated consistently.

132.

If `x` is a middle node in a doubly linked list, which code bypasses x?

- A. `x.prev.next = x.next; x.next.prev = x.prev;`
- B. `x.prev = null; x.next = null;`
- C. `head = x; tail = x;`
- D. `x.data = 0;`

Correct Answer: A

Explanation: The predecessor is linked to the successor in both directions.

133.

What is printed?

```
Node a=new Node(); a.data=5;
Node b=new Node(); b.data=6;
a.next=b; b.prev=a;
System.out.println(a.next.prev.data);
```

- A. 5
- B. 6
- C. null
- D. 0

Correct Answer: A

Explanation: a.next is b, and b.prev is a, whose data is 5.

134.

Suppose a BST contains 10 as root and 5 as its left child. Where does 4 go when inserted?

- A. In the left subtree of 5
- B. In the right subtree of 10
- C. As the right child of 5
- D. As the root

Correct Answer: A

Explanation: $4 < 10$, then $4 < 5$, so it moves left from 5.

135.

Suppose a BST contains 10 as root and 20 as its right child. Where does 30 go?

- A. In the right subtree of 20
- B. In the left subtree of 10
- C. As the left child of 20
- D. As the root

Correct Answer: A

Explanation: $30 > 10$ and $30 > 20$, so it is inserted to the right of 20.

136.

What does this inorder method print for a BST?

```
void in(Node n){
    if(n==null)return;
    in(n.left);
    System.out.print(n.data+" ");
    in(n.right);
}
```

- A. Sorted key order
- B. Insertion order always
- C. Only leaves
- D. Reverse key order always

Correct Answer: A

Explanation: Inorder traversal of a valid BST yields sorted order.

137.

What is the maximum recursion depth of inorder traversal on a tree shaped as a chain of n nodes?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: A chain has height n, so recursive depth can be n.

138.

What is the purpose of `current = current.next` in list traversal?

- A. Move to the successor node
- B. Move to the predecessor node
- C. Delete current automatically
- D. Create a new node

Correct Answer: A

Explanation: It follows the next link to the next node.

139.

A list contains 5 nodes. How many node visits can a full traversal make at most?

- A. 5
- B. 4
- C. 6
- D. 25

Correct Answer: A

Explanation: A traversal that visits every node once visits exactly 5 nodes.

140.

What should this do for an empty list?

```
Node current = head;
while(current != null) {
    System.out.println(current.data);
    current = current.next;
}
```

- A. Print nothing
- B. Throw an exception immediately
- C. Print null
- D. Create a node

Correct Answer: A

Explanation: With head null, the loop condition is false immediately.

141.

Which statement correctly updates the head when deleting the first node in a doubly linked list with more than one node?

- A. head = head.next; head.prev = null;
- B. head = head.prev; head.next = null;
- C. head = null; head.next = head;
- D. head = tail;

Correct Answer: A

Explanation: The second node becomes head and must have no predecessor.

142.

Which statement correctly updates the tail when deleting the last node in a doubly linked list with more than one node?

- A. `tail = tail.prev; tail.next = null;`
- B. `tail = tail.next; tail.prev = null;`
- C. `tail = head; tail.next = tail;`
- D. `tail = null;`

Correct Answer: A

Explanation: The previous node becomes the new tail and its next must be null.

143.

What does this code do?

```
Node p = head;
while(p != null) {
    p = p.next;
}
```

- A. Walks through the list until p becomes null
- B. Deletes every node
- C. Reverses the list
- D. Finds the middle node automatically

Correct Answer: A

Explanation: p follows each next reference until it passes the final node.

144.

For a stack used in bracket matching, why is `peek()` useful before `pop()`?

- A. It lets the algorithm verify the matching opening bracket before removing it
- B. It adds a bracket
- C. It reverses the stack
- D. It sorts brackets

Correct Answer: A

Explanation: The top must be checked for a matching opening bracket before it is removed.

145.

What is the time complexity of searching this list?

```
Node p=head;
while(p!=null && p.data != target) p=p.next;
```

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: In the worst case, all nodes may be checked.

146.

If a singly linked list has head and tail, what should tail be after inserting the first node into an empty list?

- A. The new node
- B. null
- C. The old head
- D. The previous node

Correct Answer: A

Explanation: The only node is both the first and last node, so both head and tail refer to it.

147.

If a doubly linked list's new head is `x`, what should normally be true?

- A. `x.prev == null`
- B. `x.next == null` always
- C. `x.prev == tail`
- D. x must be a leaf

Correct Answer: A

Explanation: The head has no predecessor.

148.

What is printed?

```
Node a=new Node(); a.data=1;
Node b=new Node(); b.data=2;
a.next=b;
b.next=null;
System.out.println(a.next.next);
```

- A. null
- B. 1
- C. 2
- D. a

Correct Answer: A

Explanation: a.next refers to b, and b.next was assigned null.

149.

Which condition safely traverses a linked list without dereferencing a null node?

```
Node p = head;
while (_____) {
    p = p.next;
}
```

- A. p != null
- B. p.next != null always
- C. head != null && p.next != null
- D. p == null

Correct Answer: A

Explanation: The loop must ensure p itself is not null before accessing p.next.

150.

What is printed?

```
Stack<Integer> s = new Stack<>();
s.push(5);
s.push(8);
s.pop();
System.out.println(s.peek());
```

- A. 5
- B. 8
- C. null
- D. 13

Correct Answer: A

Explanation: 8 is removed by pop, leaving 5 at the top.

LEVEL — HARD

151.

What is the output?

```
static void f(int n){
    if(n==0)return;
    System.out.print(n+" ");
    f(n-1);
    System.out.print(n+" ");
}
f(3);
```

- A. 3 2 1 1 2 3
- B. 3 2 1 3 2 1
- C. 1 2 3 3 2 1
- D. 3 1 2 1 2 3

Correct Answer: A

Explanation: The first prints happen while descending; the second set happens during unwinding.

152.

What is the key stack-space issue in this recursive traversal?

```
void walk(Node n){
    if(n==null)return;
    walk(n.left);
    walk(n.right);
}
```

For a skewed tree of n nodes, the recursion depth can be:

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$ always
- D. $O(n^2)$

Correct Answer: A

Explanation: A chain-shaped tree causes nested calls proportional to its height, which can be n.

153.

Consider:

```
int low=0, high=15;
int count=0;
while(low<=high){
    int mid=(low+high)/2;
    count++;
    if(mid==15) break;
    low=mid+1;
}
System.out.println(count);
```

What is printed?

- A. 4
- B. 3
- C. 5
- D. 1

Correct Answer: A

Explanation: The midpoints are 7, 11, 13, 14; then the interval ends without finding 15, so 4 iterations occur.

154.

A binary-search implementation uses:

```
while(low < high)
```

instead of `low <= high`. Which statement is most accurate?

- A. It can change whether a single remaining candidate is checked
- B. It always makes binary search $O(n)$
- C. It guarantees duplicates are removed
- D. It converts the search to linear search

Correct Answer: A

Explanation: The loop condition affects whether the final single-element interval is examined.

155.

What is the output?

```
Node a=new Node(); a.data=10;
Node b=new Node(); b.data=20;
Node c=new Node(); c.data=30;
a.next=b; b.next=c;
b.next = a;
Node p=a;
for(int i=0;i<4;i++){
System.out.print(p.data+" ");
p=p.next;
}
```

- A. 10 20 10 20
- B. 10 20 30 10
- C. 10 30 10 30
- D. 30 20 10 20

Correct Answer: A

Explanation: After b.next is changed to a, the reachable cycle from a is $a \rightarrow b \rightarrow a \rightarrow b$.

156.

Why is the previous code dangerous if changed to ``while(p != null)``?

- A. The list now contains a cycle, so p never becomes null
- B. The list becomes sorted
- C. p becomes head automatically
- D. Java stops after 10 iterations

Correct Answer: A

Explanation: A cycle means repeated traversal never reaches null unless a stopping condition is added.

157.

For a doubly linked list, what is wrong here when x is a middle node?

```
x.prev.next = x.next;
// forgot x.next.prev
```

- A. Backward linkage remains inconsistent
- B. The list becomes automatically sorted
- C. The head is always lost
- D. No issue exists

Correct Answer: A

Explanation: The successor's prev must also be changed to the predecessor.

158.

Which sequence correctly inserts x between a and b while preserving both directions?

- A. $x.prev=a$; $x.next=b$; $a.next=x$; $b.prev=x$;
- B. $a.next=x$; $x.prev=b$; $b.prev=a$; $x.next=a$;
- C. $x.prev=null$; $x.next=null$; $a=x$; $b=x$;
- D. $a.prev=x$; $b.next=x$;

Correct Answer: A

Explanation: The four local links are updated so $a \leftrightarrow x \leftrightarrow b$ remains consistent.

159.

A doubly linked list has head h and tail t. After deleting h when $h \neq t$, which pair is required?

- A. $head=head.next$; $head.prev=null$;
- B. $head=head.prev$; $head.next=null$;
- C. $tail=tail.prev$; $tail.next=null$;
- D. $head=null$; $tail=null$;

Correct Answer: A

Explanation: The successor becomes the head and its prev must be cleared.

160.

A doubly linked list has head h and tail t. After deleting t when $h \neq t$, which pair is required?

- A. `tail=tail.prev; tail.next=null;`
- B. `tail=tail.next; tail.prev=null;`
- C. `head=head.next; head.prev=null;`
- D. `tail=null; head=null;`

Correct Answer: A

Explanation: The predecessor becomes the tail and its next must be cleared.

161.

Suppose a singly linked list has n nodes and only a head pointer. What is the worst-case cost of appending a node?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: The final node must be located by traversing from head.

162.

Suppose a singly linked list has n nodes and both head and tail pointers. What is the usual cost of appending a node?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n^2)$

Correct Answer: A

Explanation: tail gives direct access to the final node, so only constant link updates are needed.

163.

What is the output?

```
Node n1=new Node(); n1.data=1;
Node n2=new Node(); n2.data=2;
n1.next=n2;
n2.next=null;
n2=null;
System.out.println(n1.next.data);
```

- A. 2
- B. 1
- C. null
- D. Compilation error

Correct Answer: A

Explanation: n1.next still references the node containing 2; assigning n2 null does not change n1.next.

164.

What happens here?

```
Node p=head;
head=head.next;
```

Assuming the original head had a successor, what does p reference?

- A. The original head
- B. The new head
- C. null
- D. The tail

Correct Answer: A

Explanation: p was assigned the original head before head was advanced.

165.

What does this code produce for a valid BST?

```
void in(Node n){
    if(n==null)return;
    in(n.left);
    System.out.print(n.data+" ");
    in(n.right);
}
```

If keys are 10, 5, 2, 4, 20, 30, the output is:

- A. 2 4 5 10 20 30
- B. 10 5 2 4 20 30
- C. 30 20 10 5 4 2
- D. 4 2 5 10 20 30

Correct Answer: A

Explanation: Inorder traversal of the resulting BST produces sorted key order.

166.

For the BST created by inserting 10, 5, 2, 4, 20, 30, which node is the parent of 30?

- A. 20
- B. 10
- C. 5
- D. 4

Correct Answer: A

Explanation: $30 > 10$ and $30 > 20$, so it becomes the right child of 20.

167.

For the same BST, which node is the parent of 4?

- A. 2
- B. 5
- C. 10
- D. 20

Correct Answer: A

Explanation: 4 goes left from 10, left from 5, then right from 2.

168.

What is the postorder traversal of the tree formed by inserting 10, 5, 2, 4, 20, 30?

- A. 4 2 5 30 20 10
- B. 2 4 5 10 20 30
- C. 10 5 2 4 20 30
- D. 30 20 10 5 4 2

Correct Answer: A

Explanation: Postorder is left subtree, right subtree, node: 4,2,5,30,20,10.

169.

What is the preorder traversal of the same BST?

- A. 10 5 2 4 20 30
- B. 2 4 5 10 20 30
- C. 4 2 5 30 20 10
- D. 30 20 10 5 2 4

Correct Answer: A

Explanation: Preorder visits each node before its children: 10,5,2,4,20,30.

170.

An iterative inorder implementation uses:

```
while(current != null || !stack.isEmpty())
```

Why does the `||` matter?

- A. There may be nodes waiting in the stack even when current is null
- B. It forces sorting
- C. It prevents all pushes
- D. It checks only the root

Correct Answer: A

Explanation: When current reaches null, the stack may still contain ancestors that must be processed.

171.

What would happen if iterative inorder used only:

```
while(current != null)
```

without processing the stack afterward?

- A. It could stop before visiting all nodes
- B. It would always visit every node twice
- C. It would become postorder
- D. It would sort the tree

Correct Answer: A

Explanation: current can become null while pending ancestors remain on the stack.

172.

For a balanced BST with n nodes, which complexity pair is typical for search and recursive stack space?

- A. $O(\log n)$ search and $O(\log n)$ stack depth
- B. $O(n^2)$ search and $O(1)$ stack
- C. $O(1)$ search and $O(n^2)$ stack
- D. $O(n)$ search and $O(n^2)$ stack

Correct Answer: A

Explanation: A balanced BST has logarithmic height, so path-based search and recursion depth are logarithmic.

173.

For a skewed BST with n nodes, which complexity pair is typical for search and recursion depth?

- A. $O(n)$ search and $O(n)$ recursion depth
- B. $O(\log n)$ search and $O(1)$ depth
- C. $O(1)$ search and $O(\log n)$ depth
- D. $O(n^2)$ search and $O(1)$ depth

Correct Answer: A

Explanation: A skewed tree can have height n, so a search and recursive path can both be linear.

174.

What is the most important missing update here when inserting x between a and b in a doubly linked list?

```
a.next = x;  
x.prev = a;  
x.next = b;
```

- A. b.prev = x
- B. a.prev = x
- C. x.prev = b
- D. b.next = x

Correct Answer: A

Explanation: b.prev must point back to x to preserve the backward link.

175.

What is the likely result of deleting the only node but doing only:

```
head = null;
```

in a list that also maintains `tail`?

- A. tail may incorrectly still reference the removed node
- B. The list is always correct
- C. head becomes tail automatically
- D. A new node is created

Correct Answer: A

Explanation: Both head and tail should become null for an empty list.

176.

A singly linked-list reverse usually changes each node's `next` reference. Why must a temporary reference often be used?

- A. To avoid losing the rest of the unreversed list before changing the current link
- B. To make the list sorted
- C. To create a doubly linked list automatically
- D. To avoid using head

Correct Answer: A

Explanation: The original next node must be saved before current.next is redirected.

177.

Which statement best describes why linked-list insertion after a known node can be $O(1)$ while finding that node can be $O(n)$?

- A. Link modification is constant-time once the position is known; reaching an arbitrary position may require traversal
- B. Insertion always sorts the list
- C. Finding a node uses binary search
- D. Both operations are always $O(1)$

Correct Answer: A

Explanation: The cost of locating the position and the cost of changing local links are separate.

178.

What is the output?

```
Node a=new Node(); a.data=1;
Node b=new Node(); b.data=2;
Node c=new Node(); c.data=3;
a.next=b; b.next=c;
Node p=a;
p=p.next;
System.out.println(p.data);
```

- A. 2
- B. 1
- C. 3
- D. null

Correct Answer: A

Explanation: p initially points to a; one next operation moves it to b, whose data is 2.

179.

For a doubly linked list, after executing:

```
a.next = b;
b.prev = a;
```

which relationship is established?

- A. a is immediately before b and b is immediately after a
- B. a is immediately after b
- C. a and b are unrelated
- D. a becomes the tail automatically

Correct Answer: A

Explanation: a.next points forward to b while b.prev points backward to a.

180.

What is the output?

```
Node x=new Node(); x.data=7;
Node y=new Node(); y.data=9;
x.next=y;
System.out.println(x.next.data);
x.next=null;
System.out.println(x.next);
```

- A. 9 followed by null
- B. 7 followed by 9
- C. null followed by 9
- D. 9 followed by 7

Correct Answer: A

Explanation: Initially x.next refers to y, so its data is 9. After x.next is set to null, printing it gives null.

■ IMPORTANT THINGS TO REMEMBER — DETAILED REVISION

1. Complexity

Time complexity describes how the amount of work grows as input size n grows. **Space complexity** describes how extra memory usage grows. For asymptotic analysis, ignore constant factors and lower-order terms: for example, $3n + 5$ is $\Theta(n)$. O gives an asymptotic upper bound, Ω a lower bound, and Θ a tight bound. Also distinguish **best**, **average**, and **worst case**; they describe different input situations for the same algorithm.

2. Linear Search

Linear search checks elements sequentially until the target is found or the data ends. It **does not require sorted data**. Best case is $O(1)$ when the first element matches; worst case is $O(n)$ when the target is last or absent. Remember: simple and flexible, but the number of checked elements can grow linearly.

3. Binary Search

Binary search uses the ordering of sorted data to eliminate about half of the remaining search range after each comparison. Its typical worst-case time is $O(\log n)$. Track **low**, **high**, and **mid** carefully and make sure the loop condition does not accidentally skip the final candidate. The major idea is **divide the remaining search space by two**.

4. Recursion & Stack Unwinding

Every recursive solution needs a correct **base case** and a recursive step that moves toward it. Each active call has its own stack frame. During **unwinding**, calls return from the deepest active call back toward the original caller. In tree recursion, the maximum recursion depth is related to tree height: a balanced tree can have $O(\log n)$ depth, while a skewed tree can reach $O(n)$.

5. Stack

A stack follows **LIFO — Last In, First Out**. **push()** adds an element, **pop()** removes and returns the top element, and **peek()** reads the top without removing it. Always consider empty-stack/underflow cases. For balanced brackets, push every opening bracket and, for a closing bracket, verify it matches the **most recent unmatched opening bracket** before popping.

6. Binary Tree

A binary-tree node has at most **two children**: left and right. The **root** is the topmost node and a **leaf** has no children. A complete traversal that visits every node once is typically $O(n)$. Recursive traversal uses call-stack space related to tree height, while iterative traversal can use an explicit stack.

7. Binary Search Tree (BST)

A BST adds an ordering rule to a binary tree. Under the common convention, keys smaller than a node go left and larger keys go right. Duplicates require an **explicit policy** such as placing them consistently on one side, counting them, or ignoring them. BST search/insertion follow one root-to-leaf path, so their cost depends on height: $O(\log n)$ for logarithmic height and $O(n)$ for a completely skewed tree. Inorder traversal of a valid BST gives keys in sorted order.

8. Tree Traversals

Memorize the three orders: **Preorder = Node → Left → Right**; **Inorder = Left → Node → Right**; **Postorder = Left → Right → Node**. The position of the 'visit/print' statement determines the traversal. Inorder is especially important for BSTs because it produces sorted key order. Iterative inorder uses an **explicit stack** to simulate the pending recursive calls.

9. Singly LinkedList

A normal singly linked-list node contains **data + next**. **head** refers to the first node, and the final node's 'next' is 'null' in a non-circular list. Traversal follows 'current = current.next' until 'current == null'. Insertion at the head is $O(1)$. Insertion after a known node can also be $O(1)$, because only a constant number of links change. Searching or reaching an arbitrary position is generally $O(n)$.

10. Singly LinkedList — Important Pointer Updates

To insert 'newNode' at the beginning: 'newNode.next = head; head = newNode'. To delete the first node: 'head = head.next'. To insert 'x' after 'a': 'x.next = a.next; a.next = x'. For reversal, save the original next node before changing the current node's 'next', otherwise the remaining list can become unreachable. If only a head reference is maintained, appending to the end is typically $O(n)$; maintaining a **tail** can make append $O(1)$.

11. Doubly LinkedList

A doubly linked-list node normally contains **data + prev + next**. 'next' moves forward and 'prev' moves backward. For a nonempty list, **head.prev == null** and **tail.next == null**. Every forward link should have a matching backward link: if 'a.next == b', then normally 'b.prev == a'. This extra 'prev' link costs memory but enables backward traversal and makes deletion of a known node easier.

12. Doubly LinkedList — Insertion & Deletion

To connect 'x' between 'a' and 'b', the essential relationships are: 'x.prev = a; x.next = b; a.next = x; b.prev = x'. For a known middle node 'x', bypass it with: 'x.prev.next = x.next; x.next.prev = x.prev'. When deleting the **head**, the new head's 'prev' must become null. When deleting the **tail**, the new tail's 'next' must become null. When deleting the only node, both head and tail must become null.

13. Edge Cases to Check in Exams